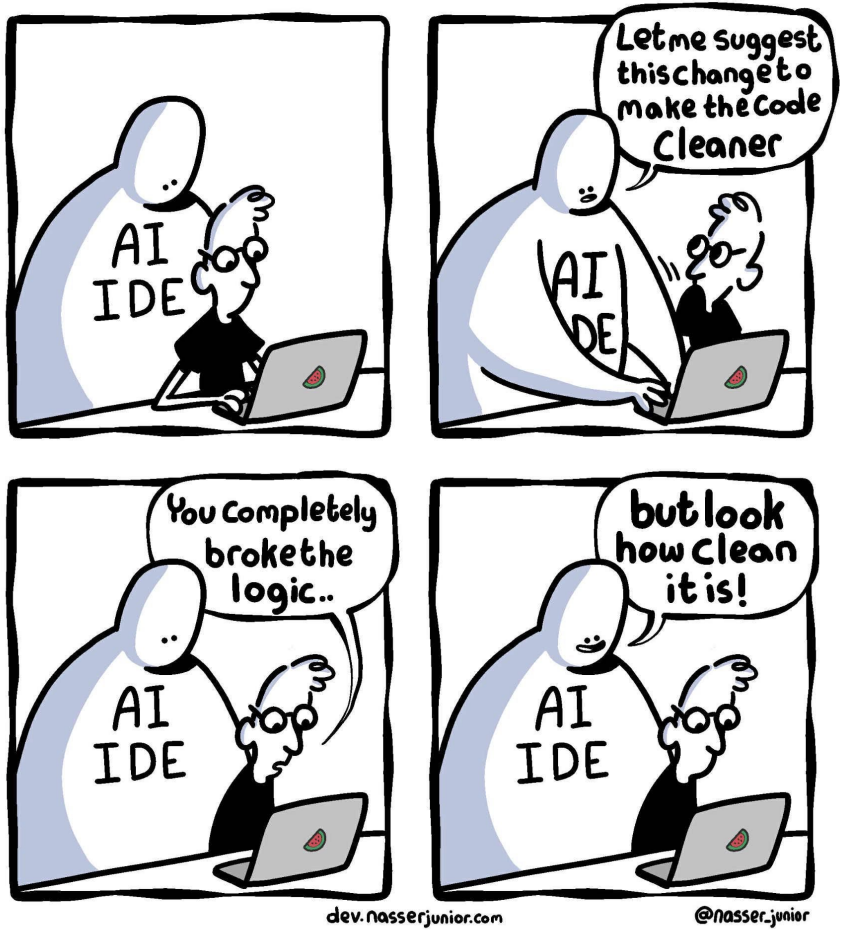


Programming with AI

Cyril Pernet, PhD
Russ Poldrack,PhD

Intended Learning Outcomes

- Gain basic understanding of agents, LLMs and in-context learning
- Gain practical knowledge of agentic programming

Have you heard the term ‘vibe coding’?
Can you tell me what it means

Use plain language to give the big picture outcome and let agents built it

Andrej Karpathy, 2025

Programing vs coding (again)

Why vibe coding does not really work if you do not know programming?

‘’Programming is the art of specifying the problem precisely enough that a computer can solve the human problem successfully’ Russ Poldrack

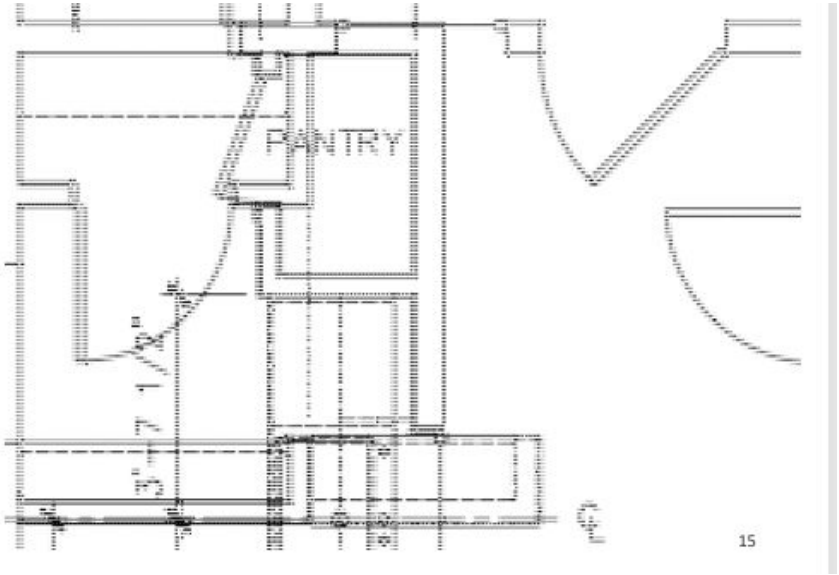
You need to be able to:

1. Handle system complexity (workflow)
2. Evaluate code quality (design, readability)
3. Debug hidden errors (unit testing)

Why specification matters?

“Put the island right over there, about three feet across from the stove. Put the pantry down at the end and another one on the other side. Then put the refrigerator next to that.”

Why specification matters?



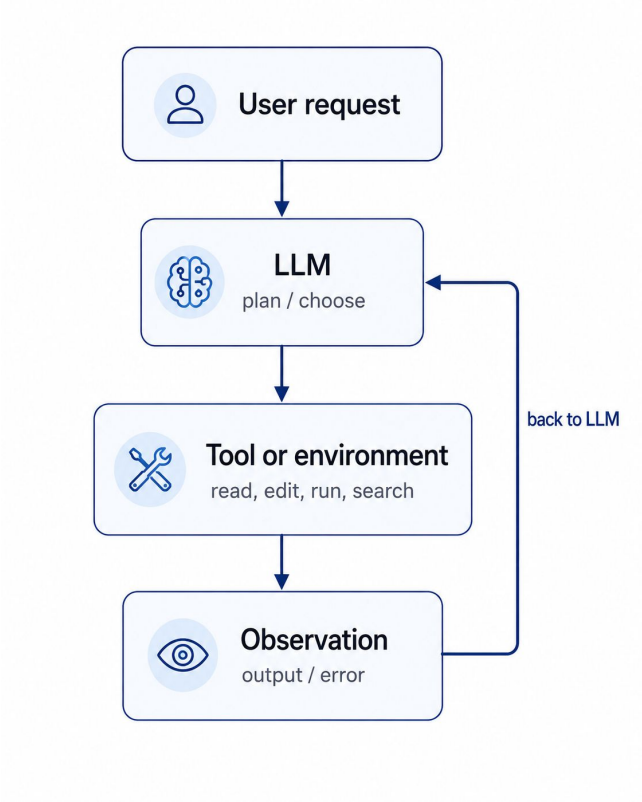
Agentic coding with Large Language models

A **coding agent** adds an external control system around the LLM.

Agents read and edit files, run terminal commands, execute tests, inspect Git diffs; consult documentation

The LLM predicts useful next tokens

The Agent uses the LLM to select and evaluate actions over multiple steps



LLM are probabilistics

An LLM is a model trained to estimate:

$P(x_t \mid x_1, \ldots, x_{t-1})$

That is, given the previous item, it predicts a probability distribution over the next item. It does not initially generate an entire answer or program. It repeatedly:

- reads the tokens already present;
- calculates probabilities for the next token;
- selects one token;
- appends it to the sequence;
- repeats.

What comes next?

for participant in participants:
    results.append(

GPT5.6 and Claude sonnet 4.6

```
for participant in participants:
   results.append(participant)
```

Clause sonnet 5

```
results = []
for participant in participants:
    results.append({
        "participant": participant,
        "value": compute_value(participant)  # replace with your actual computation
    })
```

LLM are probabilistics over tokens

LLMs use tokens, bits of words and text

‘participant_score’ may be broken down to ‘participant’, ‘_’, ‘score’

the model has no built-in abstract syntax tree unless one is supplied through tools or learned indirectly from code patterns, and that’s where being consistent in your code, how you call variables, casing, etc helps the model to read your code (see previous lecture)

Subword tokenization was developed to represent rare and unseen words using reusable smaller units rather than requiring every possible word to exist in a fixed vocabulary.

LLM pick the next probable answer on the fly

Most conversational and coding LLMs are autoregressive decoder-style models. During generation, a token can attend to previous tokens but not future tokens.

```
def mean(values):
    if not values:
        return
```

The model does not see the continuation until it generates it.

While this allows the model to revise its internal interpretation of earlier text while computing the next token; it also explains why early mistakes can propagate through a generated function. Once an incorrect assumption has been written, later tokens are conditioned on it.

In-context learning

When you interact with an LLM, the model itself is not changing

- The ability for the model to adapt to inputs within a chat session is due to “in-context learning” (context is like working memory)
- The context is the window that contains all of the information for the session
- The model generates its output conditioned on all of the information in the context window
- This means that one needs to build up all of the necessary information within the context window
- “context engineering” has replaced “prompt engineering”

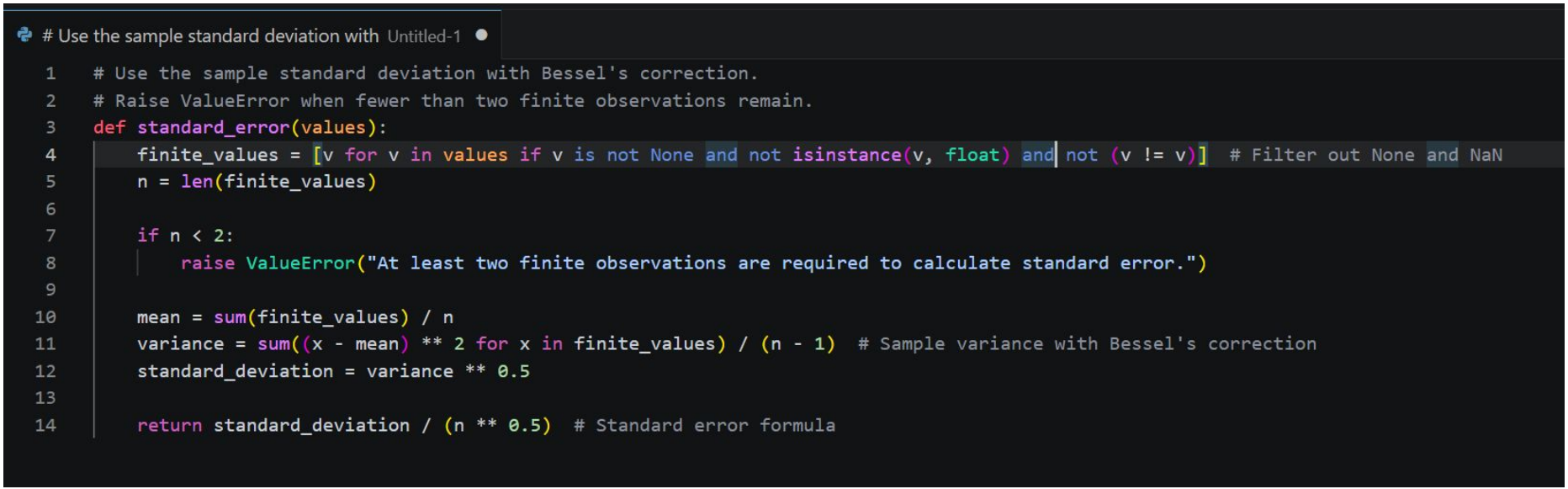
Literate programming as in context learning

def standard_error(values): Untitled-1 1

```python
def standard_error(values):
    """
    Calculate the standard error of a list of values.

    Parameters:
    values (list): A list of numerical values.

    Returns:
    float: The standard error of the values.
    """
    import numpy as np

    if len(values) == 0:
        raise ValueError("The list of values is empty.")

    mean = np.mean(values)
    std_dev = np.std(values, ddof=1)  # Sample standard deviation
    n = len(values)

    standard_error = std_dev / (n ** 0.5)
    return standard_error
```

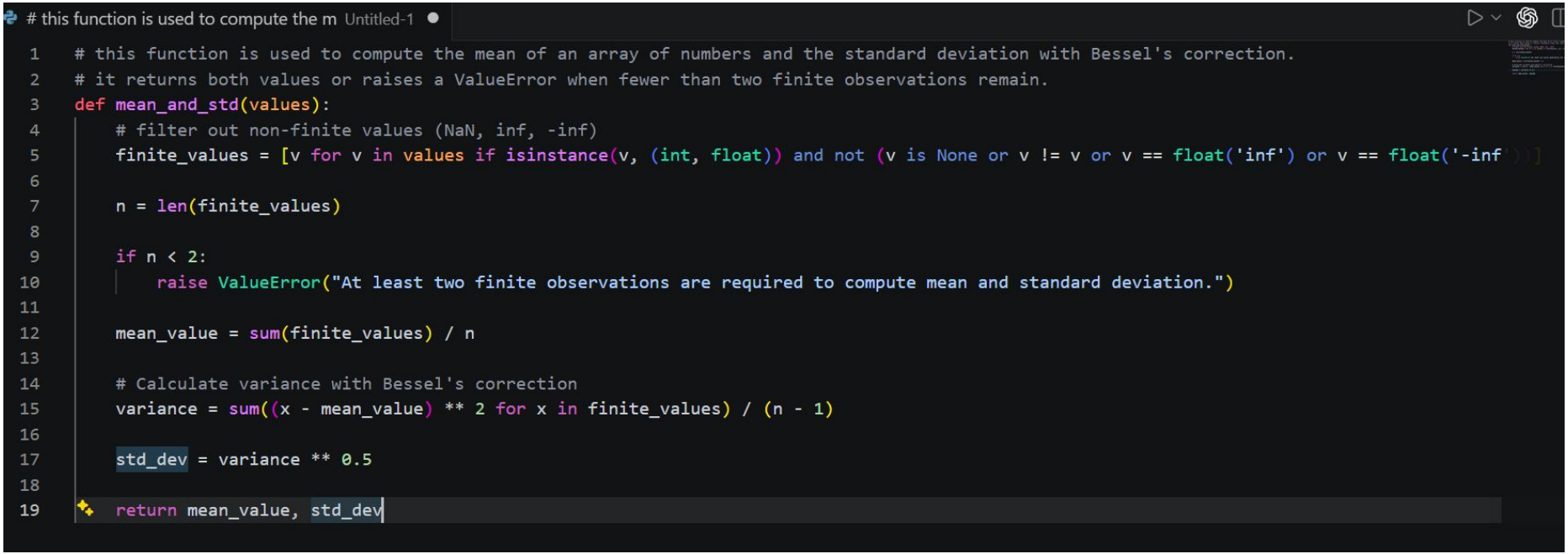
Auto Generated from
def standard_errors(values):

Literate programming as in context learning



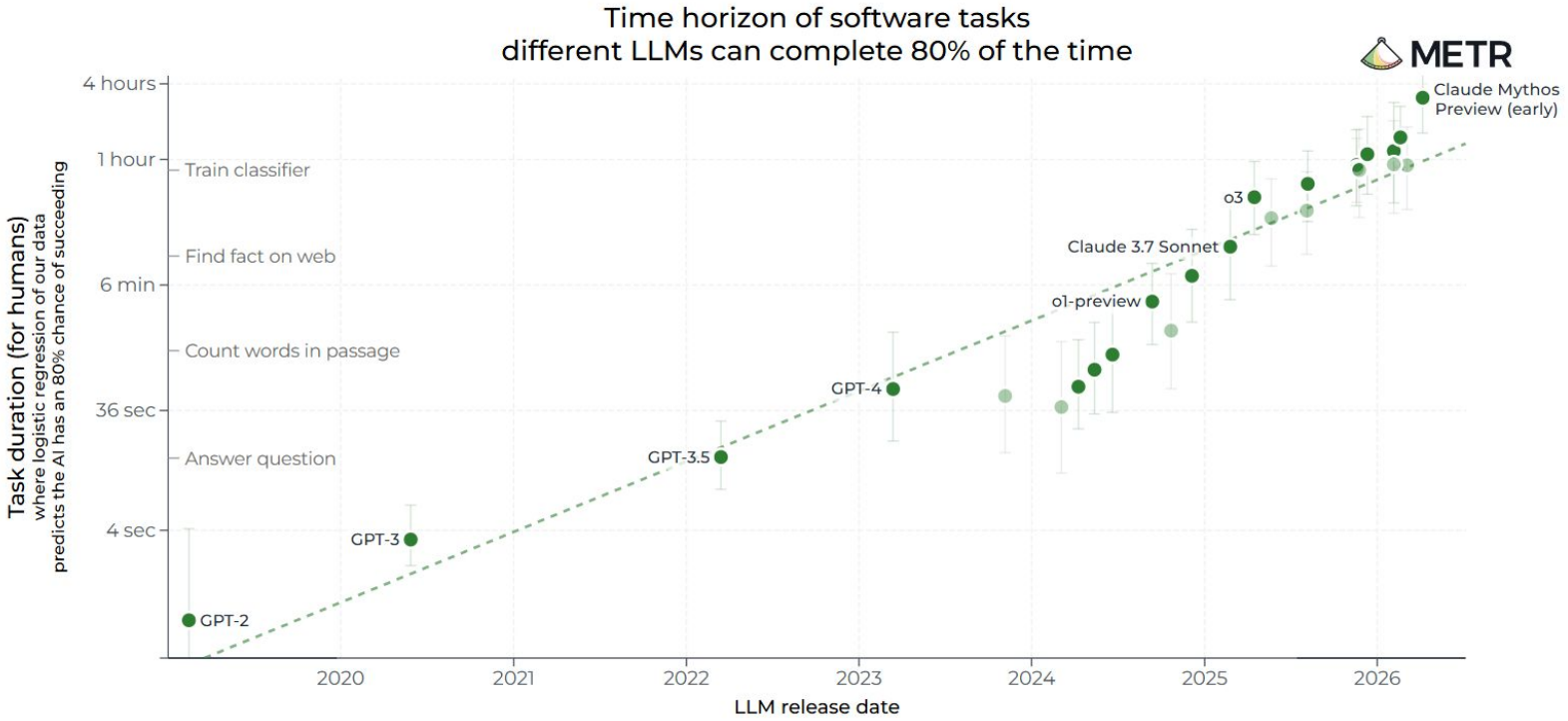
Auto Generated from
instructions and def standard_errors(values):

Literate programming as in context learning



Auto Generated from explanations alone

Measuring AI Ability to Complete Long Tasks



[Measuring AI Ability to Complete Long Tasks - METR]

Time to code → head the VS code

Prerequisite: VS code and AI agent enable

Let’s create a new folder and re-analyze Melanie's’ ice cream habits

Instructions are in the /agentic_coding repository folder

Intended Learning Outcomes

- Gain basic understanding of agents, LLMs and in-context learning
- Gain practical knowledge of agentic programming